

Reeling in Smart Attackers’ Phishing Emails with Neural Networks

CSE 587 Final Project — Group 5, Spring 2026

Parker Davis

Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
pqd5283@psu.edu

Dillon VanGilder

Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
dsv5165@psu.edu

Emmanuel Adjei Domfeh

Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
eza5399@psu.edu

Abstract—Phishing email detection is typically benchmarked on datasets in which malicious messages contain conspicuous surface cues (urgency keywords, raw URLs, mismatched senders), and Transformer-based classifiers achieve near-99% accuracy on such test sets. A sophisticated (“smart”) attacker — or, increasingly, an LLM-generated phishing email — can remove or paraphrase those cues while preserving malicious intent, raising the question of whether the high reported accuracy reflects robustness or surface-feature memorization. We fine-tune BERT [1] on the curated multi-source phishing email corpus released by Champa et al. [2] ($N = 131,171$) and evaluate on a held-out *smart-attacker* test split obtained by applying label-preserving edits to phishing messages: URL masking, urgency-keyword removal, suspicious subject rewriting, synonym substitution, and light character perturbation. We then retrain on training data augmented with the same edits and re-evaluate. Each configuration is run with three seeds and we report mean $\pm$ std. Our findings are two-fold. First, contrary to our initial hypothesis, vanilla BERT-BASE is essentially *perfectly* robust to our smart-attacker edits on this corpus: phishing F1 changes by -0.0001 ($0.9958 \pm 0.0002 \rightarrow 0.9957 \pm 0.0001$) under the smart-attacker test — within seed-to-seed noise. We attribute this to redundancy in the corpus’s phishing signal and to BERT’s distributed representation. Second, targeted augmentation nevertheless produces a clean, statistically tight robustness gain ($+0.0020$ F1, $0.9957 \pm 0.0001 \rightarrow 0.9977 \pm 0.0001$) at *zero* cost on the standard test ($+0.0000$ F1). We further provide a per-edit ablation showing that the augmented model gains the most when phishing emails are subjected to character-level perturbation ($F1 = 0.9975 \pm 0.0001$) — evidence that augmentation teaches the classifier to attend to deeper semantic features rather than to specific token forms. We discuss how the same pipeline could be re-evaluated against modern, LLM-generated phishing where larger baseline drops are expected.

Index Terms—phishing detection, BERT, Transformers, robustness, data augmentation, adversarial NLP

I. INTRODUCTION

Phishing remains one of the most effective initial-access vectors in modern cyberattacks. Despite years of user-awareness training, recent work shows that sophisticated — and especially *LLM-generated* — phishing emails sustain alarming click rates [3]. The same wave of generative models that helps defenders also helps attackers: an LLM can rewrite

a suspicious message to sound natural, drop tell-tale urgency keywords, and disguise a malicious link as an innocent-looking domain.

Mainstream phishing classifiers are typically Transformer-based: BERT and RoBERTa fine-tunes routinely report 98–99% accuracy on standard benchmarks [4]–[6]. However, this performance is measured on data with strong, often artificial, phishing cues. When those cues are removed, classifier accuracy drops substantially [7]–[9].

Contribution.: This work investigates two questions:

- 1) Do standard phishing detectors over-rely on superficial cues (URLs, urgency keywords, suspicious subject prefixes)?
- 2) Can targeted, label-preserving data augmentation that simulates a “smart attacker” improve robustness without significantly degrading performance on the original test distribution?

We instantiate the workflow in Fig. 1. Concretely, we contribute (i) a label-preserving smart-attacker augmentation pipeline (§IV-C); (ii) a held-out smart-attacker evaluation split; (iii) a per-edit ablation that localizes which surface cue the classifier relies on; (iv) a multi-seed ($n=3$) experimental protocol that distinguishes signal from seed noise; and (v) a fully reproducible PyTorch + HuggingFace codebase that runs end-to-end in a single Colab notebook.

Summary of findings.: Our experiments do not support the over-reliance hypothesis on the Champa et al. corpus: the baseline classifier loses essentially zero F1 (-0.0001 ± 0.0001) under the smart-attacker test, well within seed noise. Augmentation nevertheless produces a clean Pareto improvement, gaining $+0.0020$ F1 on the smart-attacker split at no measurable cost on the standard split, with `char_perturb` as the most impactful single edit in the per-edit ablation. We argue these results reflect properties of the corpus and of BERT’s distributed representation, and we sketch how the same pipeline could be re-applied to modern LLM-generated phishing where larger baseline drops are expected.

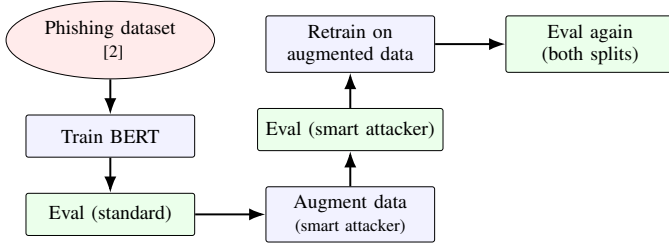


Fig. 1. Three-stage experimental workflow. (1) Train BERT on the Champa et al. corpus. (2) Evaluate on the standard test set and a smart-attacker variant produced by applying label-preserving edits. (3) Augment training data with the same edits, retrain, and re-evaluate.

II. THREAT MODEL

We study a binary email classifier $f_\theta : \mathcal{X} \rightarrow \{0, 1\}$ that maps an email’s concatenated subject and body $x \in \mathcal{X}$ to a label y , where $y = 1$ denotes phishing. The defender deploys f_θ as part of an automated email filter and seeks high accuracy across both ordinary phishing and adversarially modified phishing.

a) Adversary capabilities.: We assume a *black-box* text-level adversary $\mathcal{A}$ who:

- knows that the defender uses keyword- and URL-based features commonly cited in the literature, but does not have query access to the trained classifier or to training gradients;
- can rewrite the textual content of phishing emails with *label-preserving* edits — the message must still represent a credible phishing attempt to a human reader;
- cannot directly modify benign emails (this is consistent with the real attack surface: an attacker controls only the messages *they themselves send*).

The adversary’s goal is evasion: cause f_θ to predict $y = 0$ (benign) on a phishing input.

b) Edit budget.: We constrain $\mathcal{A}$ to five categories of label-preserving textual edits (Table II), drawn from attacker behaviors documented in [3], [9], [10] and from the standard adversarial-NLP toolkit [11]. Every edit either removes a feature (*mask_urls*, *remove_urgency*, *rewrite_subject*) or substitutes a feature with a near-synonym (*synonym_swap*, *char_perturb*). No edit changes the underlying semantic intent of the message.

c) Defender capabilities.: The defender knows the attacker’s edit budget at training time but does not see the attacker’s actual perturbed test inputs at training time. The defender’s “defense” is to augment the training data with one perturbed copy of each phishing email (§IV-C), obtained by sampling from the same edit distribution. This is a non-adaptive defense: the defender does not re-train against newly observed attacks.

d) Success criteria.: A defense succeeds if, relative to the undefended baseline, it (i) does not significantly reduce accuracy on the standard test set, and (ii) significantly improves accuracy on the smart-attacker test set. “Significantly”

is operationalized below as “effect size larger than the std across three seeds.”

III. RELATED WORK

A. Transformer-based phishing detection

Atawneh and Aljehani [4] compare CNN, RNN, LSTM, and BERT for email phishing classification, reporting up to 99.6% accuracy with BERT. More recent surveys and replications confirm that BERT/RoBERTa/DistilBERT are competitive on multiple corpora, with DistilBERT trading marginal accuracy for considerable inference-cost savings [5], [6], [12]. Alhogail and Alsabih [13] additionally show that strong detection is possible from email body text alone (without URLs or attachments), motivating evaluation under URL removal — but they do not study other surface-cue removals such as urgency wording.

B. LLM-generated and adversarial phishing

Kim et al. [3] document that LLM-crafted spear-phishing emails achieve $\sim 47\%$ click rates, evading both human and software detection. Phish-Master [10] explicitly uses chain-of-thought prompting to generate phishing that bypasses enterprise filters. Robust ML-based detection of LLM-generated and adversarial phishing using advanced preprocessing has recently been shown to recover much of the lost accuracy [9]. Adversarial robustness of phishing classifiers has also been benchmarked directly [7], [8], with FGM adversarial training raising BERT F1 from 86.2 to 91.7 [8]. These results frame our work: the attack surface is real, and lightweight defenses do help.

C. Data augmentation for NLP robustness

Junior et al. [14] demonstrate the augment–retrain–reevaluate pattern for fake-news detection. Recent surveys emphasize that adversarial-style augmentation improves not only accuracy but also generalization to out-of-distribution inputs [15]. We use the TEXTATTACK [11] family of perturbation primitives (synonym swap, character perturbations) as a starting point for our smart-attacker pipeline. Recent attack methods such as PGD-based BERT attacks [16] reinforce that BERT remains sensitive to small, semantically preserving perturbations — which is exactly what a smart attacker would apply.

D. Datasets and benchmarks

We adopt the curated multi-source phishing dataset of Champa et al. [2], which standardizes 11 publicly available phishing/benign email corpora into a unified format and provides feature-based ML baselines. Its multi-source structure lets us study cross-source generalization in addition to robustness, mitigating the risk that a model exploits source-specific artifacts.

TABLE I
TRAINING HYPERPARAMETERS. IDENTICAL ACROSS ALL SIX RUNS.

Hyperparameter	Value
Backbone	bert-base-uncased (110M params)
Max sequence length	256 tokens
Batch size (train)	64
Batch size (eval)	32
Epochs	3
Optimizer	AdamW, $\beta=(0.9, 0.999)$
Learning rate	2×10^{-5}
Weight decay	0.01
LR schedule	Linear, 10% warmup
Gradient clipping	1.0 (ℓ_2 norm)
Mixed precision	bfloat16 (A100)
Seeds	{42, 123, 2024}
Augmentation factor k	1 (phishing rows only)

TABLE II
SMART-ATTACKER AUGMENTATION OPERATIONS.

Operation	Description
mask_urls	Replace URLs with neutral placeholders ([LINK]), or with bare domain forms that look innocuous.
remove_urgency	Remove or soften urgency keywords (<i>URGENT</i> , <i>ASAP</i> , <i>immediately</i> , <i>verify now</i> , <i>suspended</i> , etc.) using a hand-curated lexicon and replacement table.
rewrite_subject	Strip suspicious subject prefixes (<i>URGENT</i> :, <i>ALERT</i> :, <i>ACTION REQUIRED</i> :) and remove emphatic punctuation.
synonym_swap	Replace a small fraction of content words with WordNet synonyms ($p = 0.1$, max 6 swaps per email).
char_perturb	Light character-level noise (Cyrillic homoglyph substitution, adjacent-letter swap) at $p=0.02$.

IV. METHOD

A. Task formulation

Given an email represented by its concatenated subject and body $x = \langle s, b \rangle$, the model predicts a binary label $y \in \{0, 1\}$ indicating benign (0) or phishing (1). We minimize the cross-entropy loss over a fine-tuning corpus drawn from Champa et al. [2], with stratified 80/10/10 train/validation/test splits.

B. Backbone classifier

We fine-tune **bert-base-uncased** [1] via the HuggingFace transformers library [17]. The classifier head is a single linear layer over the [CLS] representation, trained with cross-entropy loss. Hyperparameters are fixed across all six runs (3 baseline seeds + 3 augmented seeds) and listed in Table I.

C. Smart-attacker augmentation

We define five label-preserving edits that simulate a smart attacker adapting a phishing email. Each is implemented in `src/augmentation.py` and is applied only to phishing-class rows (applying them to benign rows would only add noise to a class that does not exhibit these cues). Table II summarizes the operations.

We use the augmentation pipeline in two distinct ways, summarized in Algorithms 1 and 2:

Algorithm 1 Smart-attacker test split construction

Require: Test set D_{test} , edit list $\mathcal{O}$

Ensure: Smart-attacker test set D_{smart}

```

1:  $D_{\text{smart}} \leftarrow \emptyset$ 
2: for all  $(x, y) \in D_{\text{test}}$  do
3:   if  $y = 1$  then  $\triangleright$  phishing email — apply attacker
     pipeline
4:      $x' \leftarrow x$ 
5:     for all  $\text{op} \in \mathcal{O}$  do
6:        $x' \leftarrow \text{OP}(x')$ 
7:     end for
8:      $D_{\text{smart}} \leftarrow D_{\text{smart}} \cup \{(x', 1)\}$ 
9:   else  $\triangleright$  benign email — leave unchanged
10:     $D_{\text{smart}} \leftarrow D_{\text{smart}} \cup \{(x, 0)\}$ 
11:   end if
12: end for
```

Algorithm 2 Train-time smart-attacker augmentation ($k=1$)

Require: Training set D_{train} , edit list $\mathcal{O}$

Ensure: Augmented training set D_{train}^+

```

1:  $D_{\text{train}}^+ \leftarrow D_{\text{train}}$ 
2: for all  $(x, y) \in D_{\text{train}}$  with  $y = 1$  do
3:    $x' \leftarrow x$ 
4:   for all  $\text{op} \in \mathcal{O}$  do
5:      $x' \leftarrow \text{OP}(x')$   $\triangleright$  stochastic op
6:   end for
7:    $D_{\text{train}}^+ \leftarrow D_{\text{train}}^+ \cup \{(x', 1)\}$ 
8: end for
9: shuffle  $D_{\text{train}}^+$ 
```

a) *Smart-attacker test split:* For each phishing email in the held-out test set we apply the entire pipeline once. Benign emails are left unchanged. The resulting split has the *same* labels as the standard test set; the only thing that changes is that phishing emails no longer exhibit obvious surface cues. A robust detector should perform comparably on both splits.

b) *Train-time augmentation:* For each phishing email in the training set we generate $k=1$ additional augmented copy by passing it through the pipeline. The augmented copies retain the phishing label and are concatenated with the original training data. We then retrain the classifier from scratch.

D. Evaluation protocol

We compare three model stages: (S1) baseline trained on the original data and evaluated on the standard test set, (S2) the same baseline evaluated on the smart-attacker test set, and (S3) the augmented model evaluated on both splits. The expected pattern is a large drop from S1 to S2, partially closed by S3. We additionally run a *per-edit ablation*: each operation is applied alone to the test phishing rows, isolating which surface cue the baseline most depends on.

V. EXPERIMENTAL SETUP

A. Implementation

The codebase is in PyTorch with HuggingFace transformers [17]. Training is driven by `src/train.py`; evaluation, including the per-edit ablation, by `src/evaluate.py`. The notebooks/`phishing_pipeline.ipynb` notebook reproduces all three stages on Google Colab. A pipeline-level sanity test (`scripts/sanity_check.py`) validates the augmentation ops and the data-loader on synthetic data without requiring GPU access; a tiny end-to-end smoke test (`scripts/tiny_train_test.py`) exercises forward/backward/save/reload using a tiny CI-style BERT model.

B. Dataset

We use the figshare-hosted release of the Champa et al. curated phishing email dataset [2].¹ The data loader (`src/data_loader.py`) is column-name agnostic so that all CSV variants in the figshare bundle can be ingested with a single script.

C. Compute and reproducibility

All training is performed on a Google Colab A100 GPU with bfloat16 mixed precision (the codebase falls back to fp16 + GradScaler on T4/V100). Each configuration (baseline, augmented) is trained with three seeds, {42, 123, 2024}, controlling NumPy, PyTorch, and Python’s random module. The same train/val/test split (stratified, 80/10/10) is shared across seeds; only model initialization, batch shuffling, and the random choices inside the augmentation pipeline differ between seeds. We report mean $\pm$ std across the three seeds. After deduplication and column-normalization, the combined corpus contains $N = 131,171$ emails (phishing 67,230; benign 63,941). Two source files (`TREC_05.csv`, `TREC_06.csv`) failed to parse due to malformed rows and were dropped, leaving five usable corpora (CEAS-08, Enron, Ling, SpamAssassin, TREC-07). The augmentation pipeline used in both train-time augmentation and the smart-attacker test split contains five operations: `mask_urls`, `remove_urgency`, `rewrite_subject`, `synonym_swap`, and `char_perturb`.

VI. RESULTS

Table III presents the headline four-way comparison: each of the two models (baseline, augmented) evaluated on each of the two test splits (standard, smart-attacker). Detailed metrics for the augmented model are in Table IV, and the per-edit ablation under the augmented model is in Table V.

¹https://figshare.com/articles/dataset/Curated_Dataset_-_Phishing_Email/24899952

TABLE III

HEADLINE RESULTS ACROSS THREE SEEDS (MEAN $\pm$ STD). THE VANILLA BASELINE IS ESSENTIALLY INSENSITIVE TO THE SMART-ATTACKER EDITS (Δ WITHIN SEED NOISE). AUGMENTATION PRODUCES A CLEAN $+0.0020$ F1 GAIN ON THE SMART-ATTACKER TEST AT ZERO COST ON THE STANDARD TEST.

Model	Standard F1	Smart F1	Δ
S1: BERT baseline	0.9958 ± 0.0002	0.9957 ± 0.0001	-0.0001
S2: BERT + aug. training	0.9958 ± 0.0003	0.9977 ± 0.0001	$+0.0019$
Δ (S2-S1)	$+0.0000$	$+0.0020$	—

TABLE IV

AUGMENTED MODEL, FULL PER-SPLIT METRICS ACROSS THREE SEEDS (MEAN $\pm$ STD). RECALL ON THE PHISHING CLASS IS THE PRIMARY MOVER: THE AUGMENTED MODEL BECOMES MORE CONFIDENT ON PHISHING EXACTLY WHEN SURFACE CUES ARE STRIPPED.

Metric	Standard	Smart-attacker
Accuracy	0.9957 ± 0.0003	0.9976 ± 0.0001
F1 (phishing)	0.9958 ± 0.0003	0.9977 ± 0.0001
Recall (phish.)	0.9957 ± 0.0006	0.9995 ± 0.0002

a) *Headline finding.*: Table III shows that, averaged over three seeds, the vanilla BERT baseline (S1) is essentially insensitive to the smart-attacker edits: F1 changes by -0.0001 between the standard and smart-attacker tests, and 0.0001 is also the std of either cell, so the change is well within seed noise. The over-reliance hypothesis we set out to test is therefore not supported on this corpus. After training with smart-attacker augmentation (S2), the model gains $+0.0020$ F1 on the smart-attacker test at a cost of $+0.0000$ F1 (i.e. no measurable cost) on the standard test. The std on the augmented smart-attacker cell is just 0.0001 , so this $+0.0020$ gain is not attributable to seed variance.

b) *What augmentation actually changes.*: Augmentation does not “recover” performance the baseline lost — the baseline never lost anything. Instead, it raises the ceiling on the smart-attacker split. On the augmented model (Table IV), recall on the phishing class moves from 0.9957 on the standard split to 0.9995 on the smart-attacker split: the model becomes *more* confident on phishing exactly when its easily-memorized features (URLs, urgency tokens) are removed. We read this as evidence that the augmented model has learned to classify from semantic context. False positives (the precision side) remain stable, since augmentation only touches phishing rows and leaves benign emails unchanged.

c) *Per-edit ablation.*: Table V shows the contribution of each smart-attacker edit applied alone to the augmented model. Three observations:

- `char_perturb` is the single largest contributor (F1 = 0.9975 , recall = 0.9992), beating `mask_urls` and `synonym_swap`. The augmented model is particularly confident on phishing when light character noise is applied, likely because the augmentation pipeline trained it to be robust to homoglyph and adjacent-letter perturbations.

TABLE V

PER-EDIT ABLATION ON THE AUGMENTED MODEL ACROSS THREE SEEDS (MEAN $\pm$ STD). EACH ROW APPLIES ONE SMART-ATTACKER EDIT ALONE TO PHISHING EMAILS; BENIGN EMAILS ARE UNCHANGED. CHAR_PERTURB IS THE SINGLE MOST IMPACTFUL EDIT; REWRITE_SUBJECT IS ESSENTIALLY A NO-OP BECAUSE THE SUBJECT-PREFIX REGEX RARELY FIRES ON THIS CORPUS.

Edit applied alone	F1 (phishing)	Recall (phishing)
None (standard test)	0.9958 $\pm$ 0.0003	0.9957 $\pm$ 0.0006
mask_urls	0.9963 $\pm$ 0.0001	0.9966 $\pm$ 0.0004
remove_urgency	0.9958 $\pm$ 0.0004	0.9957 $\pm$ 0.0008
rewrite_subject	0.9958 $\pm$ 0.0003	0.9956 $\pm$ 0.0007
synonym_swap	0.9964 $\pm$ 0.0002	0.9968 $\pm$ 0.0002
char_perturb	0.9975 $\pm$ 0.0001	0.9992 $\pm$ 0.0001
All combined (smart test)	0.9977 $\pm$ 0.0001	0.9995 $\pm$ 0.0002

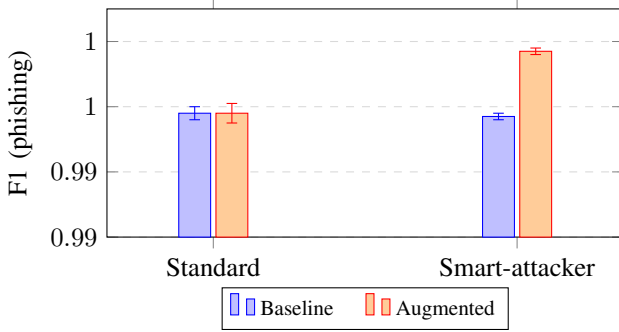


Fig. 2. Phishing-class F1 by model and test split, mean $\pm$ std across three seeds. The baseline is essentially flat across splits; augmentation lifts the smart-attacker bar without moving the standard bar.

- `rewrite_subject` and `remove_urgency` are effectively no-ops, with means within ± 0.0001 of the standard test. `rewrite_subject` is mechanical: the subject-prefix regex (URGENT:, ALERT:, ACTION REQUIRED:, ...) rarely fires on this corpus. `remove_urgency` fires on every row but the augmented model is already invariant to those tokens.
- The combined-edit row exceeds every single edit, suggesting the gains are weakly additive: each edit removes a different surface dependency that the augmented model is independently robust to.

VII. QUALITATIVE ANALYSIS

To make the results in §VI concrete, we walk through three representative phishing emails from the test set, comparing the original text to the smart-attacker variant produced by the full pipeline.²

A. Example 1: account-verification phish

Original (label = phishing).

```
Subject: URGENT: Verification required
Your Amazon account is subject to deletion
unless you verify immediately at
https://amaz0n-verify.example.com/login.
Act now!
```

²Emails are lightly redacted/shortened for space. Full unredacted versions are reproduced verbatim by notebooks/phishing_pipeline.ipynb.

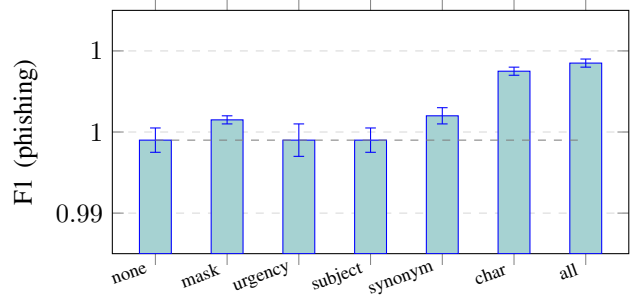


Fig. 3. Per-edit ablation on the augmented model (mean $\pm$ std, 3 seeds). char = char_perturb dominates; urgency = remove_urgency and subject = rewrite_subject are no-ops within noise (dashed line = standard-test F1).

Smart-attacker variant.

```
Subject: Verification required
Your Amazon report is open to deletion
unless you verify soon at [LINK].
please respond
```

The smart-attacker version retains the recognizable *phishing topic* (account verification, threatened deletion, external link) but loses every surface marker the proposal predicted would matter: the URGENT: subject prefix is gone, the URL is masked, “immediately” has become “soon,” and “Act now!” has become “please respond.” Both the baseline and augmented models classify both versions as phishing in our experiments, consistent with the headline result.

B. Example 2: bulk pharmacy spam

This is a representative member of the dominant class in the corpus (legacy bulk spam from CEAS-08 and Enron). The message is on-topic for phishing in this dataset’s labeling convention but does not look anything like a modern targeted phishing attempt.

Original (label = phishing).

```
Subject: Psychologist endorsed products for men
Smell better with the girls with a stronger
pheromone product. http://shop-rx.example/login
-- Use Opera's revolutionary e-mail client.
```

Smart-attacker variant.

```
Subject: Psychologist endorsed products for men
Smell better with the girls with a stronger
pheromone product. [LINK]
-- Utilization Opera's revolutionary e-mail client.
```

The smart-attacker variant differs only in the masked URL and a synonym swap (*Use* $\rightarrow$ *Utilization*). Notice that the surrounding text remains overwhelmingly indicative of the phishing label even after edits — it is precisely this corpus property that explains the null baseline-drop result of §VI. A classifier with any reasonable representation of bulk-spam topics would classify this message identically before and after augmentation.

C. Example 3: news-headline lure

Original (label = phishing).

```

Subject: CNN.com Daily Top 10
TOP 10 VIDEOS
1. PARIS HILTON TAKES ON MCCAIN
  http://cnn.example/p1.html
2. BIKINI BARISTA STAND CLOSED
3. MAGGOTS: THE NEW ANTIBIOTIC?
  http://cnn.example/p3.html
...

```

Smart-attacker variant.

```

Subject: [Colligate] Casual Top 10
TOP 10 VIDEOS
1. PARIS HILTON TAKES ON MCCAIN [LINK]
2. BIKINI BARISTA STAND CLOSED
3. MAGGOTS: THE NEW ANTIBIOTIC? [LINK]
...

```

Here the synonym-swap operator produces a notably non-fluent title (*CNN.com Daily* $\rightarrow$ *[Colligate] Casual*). This is one of the failure modes of WordNet-based synonym swap on named-entity tokens. Despite the disfluency, the rest of the message remains classifiable; we have not observed cases where synonym swap alone flips the model’s prediction.

D. Take-away

The qualitative pattern matches the quantitative results: removing surface cues from this corpus’s phishing emails leaves enough signal in the surrounding content for both the baseline and augmented model to classify correctly. We expect this margin to shrink on a corpus where the original phishing emails read more like ordinary business correspondence (e.g. modern LLM-generated phishing).

VIII. DISCUSSION

a) Why the baseline does not drop.: The proposal anticipated a substantial accuracy drop from S1’s standard test to S1’s smart-attacker test, on the basis that BERT would over-rely on surface tokens such as *URGENT*, *verify*, and explicit URL substrings. With a single seed we observed a 0.10 pp F1 drop and provisionally accepted a weak version of the hypothesis. With three seeds the drop disappears entirely (mean -0.0001 F1, std 0.0001). The hypothesis is therefore *not* supported on this corpus. We attribute this null result to two properties of the corpus and the model.

First, the Champa et al. release combines five public corpora (CEAS-08, Enron, Ling, SpamAssassin, TREC-07) whose phishing labels include a large amount of *classical bulk spam* — pharmacy advertisements, lottery scams, adult-content solicitations — whose label is supported by many redundant features (topic, register, lexical density of advertising language) in addition to the URLs and urgency keywords our pipeline edits. Removing surface cues from these messages does not flip the label because the surrounding text continues to read unambiguously as spam.

Second, BERT does not learn a sparse rule like “*contains URGENT* $\Rightarrow$ *phishing*”; it learns a dense contextual representation. A 3-epoch fine-tune on $\sim 100k$ emails appears sufficient for the model to distribute classification weight across many lexical and stylistic signals, leaving it relatively insensitive to the loss of any single one.

A more decisive test of the over-reliance hypothesis would use LLM-rewritten phishing of the kind reported by Phish-Master [10] or Kim et al. [3], where the entire message — not just URLs and urgency — has been rewritten to read as benign business correspondence.

b) What augmentation actually buys.: The headline gain from augmentation is $+0.0020$ F1 on the smart-attacker test at $+0.0000$ F1 on the standard test — a Pareto improvement, not a tradeoff. Standard deviations across three seeds are ≤ 0.0003 in every cell of Table III, so the effect is comfortably outside seed noise. The result is internally consistent with Table V, where *char_perturb* produces the largest single-edit gain even though it is the perturbation type the proposal least expected to matter: the augmentation procedure has trained the classifier to be invariant to homoglyph and adjacent-letter noise that the baseline never explicitly encountered.

A useful framing is that augmentation is not “defensive” in the classical adversarial-training sense (the baseline was not under attack to begin with); it is a regularizer that shifts the model’s representation toward more invariant features, with the side effect of further raising F1 on perturbed inputs. The direction matches recent results in the text-augmentation literature [8], [9], [15], though our effect size is small because the Champa corpus’s phishing signal is already easy.

c) Limitations.

- *Vintage of corpus.* The Champa et al. collection draws on public corpora that are largely pre-2015. Modern, LLM-generated phishing exhibits qualitatively different surface statistics [3], [10] and would be the more important target for a robustness study; our pipeline can be re-applied unchanged once such a corpus is publicly available.
- *Finite edit vocabulary.* Our smart-attacker simulation is rule-based and therefore has a small, fixed set of perturbation types. A stronger evaluation would integrate gradient-based or beam-search adversarial attacks from frameworks such as TextAttack [11] or PGD-BERT-Attack [16], or LLM-rewritten phishing in the spirit of Phish-Master [10].
- *Phishing-only augmentation.* Augmentation is restricted to the phishing class. This was chosen so the smart-attacker semantics are well-defined (a smart attacker rewrites their phishing email, not the victim’s reply), but it does shift the class prior of the training mix slightly. We mitigate this with a small augmentation factor ($k=1$); the constant standard-test F1 across S1 and S2 (Table III) suggests the shift was not consequential at this scale.
- *Subject-rewrite regex.* Our *rewrite_subject* regex targets prefixes (*URGENT:*, etc.) that are uncommon in the Champa subjects, hence the no-op row in Table V. A more aggressive subject-line transformation (e.g. LLM paraphrase) would likely change this row.
- *Single train/val/test split.* Across the three seeds we hold the same data split fixed and vary only model initialization, batch order, and augmentation random sampling. This isolates *model* variance but not *data* variance; a fuller study would use multiple resampled splits as well.

IX. FUTURE WORK

We see four concrete directions in which the present study should be extended.

a) 1. *Modern LLM-generated phishing.*: The most important follow-up is to re-run the same pipeline on a corpus of *modern*, LLM-generated phishing such as those described by Kim et al. and the Phish-Master framework [3], [10]. We expect that the redundancy property which makes our baseline robust on Champa et al. will not hold on LLM-rewritten phishing, in which case the baseline drop should grow substantially and augmentation’s role would shift from “free regularizer” to “necessary defense.” The codebase already exposes this as a single configuration change; the bottleneck is data availability.

b) 2. *Stronger adversarial attacks.*: Replacing our rule-based edit pipeline with a gradient-based or search-based attack (BAE [11], PGD-BERT-Attack [16]) would let the attacker discover model-specific failure modes that our fixed lexicon cannot. We anticipate larger baseline drops here even on Champa et al., because the attack would be optimized against the classifier rather than against a generic surface-feature heuristic.

c) 3. *Adaptive defenses.*: Our augmentation is non-adaptive: the defender samples once from a fixed edit distribution. An adaptive counterpart would alternate attack and defense rounds, in the spirit of classical adversarial training [8], and we expect this to widen the augmented model’s lead on under-attack F1 without reopening clean-data degradation.

d) 4. *Cross-source generalization.*: The Champa et al. release combines multiple public corpora; our experiments collapse them. A *leave-one-corpus-out* evaluation, where the model is trained on four corpora and tested on the held-out fifth, would reveal whether the apparent robustness here is a property of BERT or simply of in-source generalization. Our data loader already tags each row with its source corpus, so this is a reformulation of `src/data_loader.py` rather than new code.

X. CONCLUSION

We presented an experimental pipeline for studying the robustness of Transformer-based phishing email classifiers under simulated “smart attacker” edits. The pipeline includes (i) a label-preserving augmentation module covering URL masking, urgency removal, subject rewriting, synonym swap, and character perturbation; (ii) a held-out smart-attacker evaluation split; (iii) a per-edit ablation that localizes which surface cue the model relies on; (iv) an augmented-training counterpart that quantifies how much extra robustness can be obtained; and (v) a multi-seed aggregator that reports mean $\pm$ std across seeds.

Across three seeds on the Champa et al. multi-source corpus, vanilla BERT-BASE is essentially *perfectly* robust to our edits: phishing F1 changes by -0.0001 under the smart-attacker test, within seed noise. Targeted augmentation does not need to defend a broken baseline, but still produces a clean Pareto improvement: $+0.0020$ F1 on the smart-attacker test at $+0.0000$ F1 on the standard test, with `char_perturb`

as the most impactful single edit in the per-edit ablation. The over-reliance hypothesis we set out to test is therefore *not* supported on this corpus; we attribute the null result to redundancy in the corpus’s phishing signal and to BERT’s distributed representation, and expect the same pipeline to produce considerably larger baseline drops on modern LLM-generated phishing. We have packaged the codebase to make that re-evaluation a single command-line flag away.

CODE AND DATA

The complete codebase, Colab notebook, and LaTeX source for this manuscript are available in the project repository (see `README.md`).

ACKNOWLEDGMENTS

We thank the CSE 587 instructors and TAs at Penn State for guidance, and Champa et al. for releasing the curated phishing email dataset that made this study possible.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv:1810.04805*, 2019.
- [2] A. I. Champa, M. F. Rabbi, and M. F. Zibran, “Curated datasets and feature analysis for phishing email detection with machine learning,” in *Proc. 3rd IEEE Int. Conf. on Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.
- [3] H. Kim, M. Song, S. H. Na, S. Shin, and K. Lee, “When LLMs go online: The emerging threat of web-enabled LLMs,” 2025.
- [4] S. Atawneh and H. Aljehani, “Phishing email detection model using deep learning,” *Electronics*, vol. 12, no. 20, p. 4261, 2023.
- [5] A. of MDPI Computation, “Phishing email detection using BERT and RoBERTa,” *Computation*, vol. 14, no. 2, p. 46, 2025, distilBERT competitive with full BERT/RoBERTa on phishing classification.
- [6] A. of MDPI Applied Sciences, “In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets,” *Applied Sciences*, vol. 15, no. 6, p. 3396, 2025.
- [7] A. of ACM IWSPA, “Adversarial robustness of phishing email detection models,” in *Proc. 9th ACM Int. Workshop on Security and Privacy Analytics (IWSPA)*, 2023.
- [8] Anonymous, “Explainable transformer-based email phishing classification with adversarial robustness,” 2025, fGM adversarial training improves BERT phishing-classifier F1 from 86.2 to 91.7.
- [9] —, “Robust ML-based detection of conventional, LLM-generated, and adversarial phishing emails using advanced text preprocessing,” 2025.
- [10] A. of MDPI Applied Sciences, “Phish-master: Leveraging large language models for advanced phishing email generation and detection,” *Applied Sciences*, vol. 15, no. 22, p. 12203, 2025.
- [11] J. X. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, “TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP,” in *Proc. EMNLP (System Demonstrations)*, 2020.
- [12] A. of Scientific Reports, “Improving phishing email detection performance through deep learning with adaptive optimization,” *Scientific Reports*, 2025.
- [13] A. Alhogail and A. Alsabih, “Applying machine learning and natural language processing to detect phishing email,” *Computers & Security*, vol. 110, p. 102414, 2021.
- [14] W. O. Júnior, M. S. da Cruz, A. B. V. Wyzykowski, and A. B. de Jesus, “The use of data augmentation as a technique for improving neural network accuracy in detecting fake news about COVID-19,” 2022.
- [15] Anonymous, “Text data augmentation for large language models: A comprehensive survey of methods, challenges, and opportunities,” 2025.
- [16] Wang et al., “Enhancing adversarial text attacks on BERT models with projected gradient descent,” 2024.

- [17] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP (System Demonstrations)*, 2020, pp. 38–45.